1. Explain your project.

Answer:

“My project is called **SkyPredict AI**. It is an end-to-end machine-learning application for flight analytics. The main objective is to use historical flight and passenger data to provide flight-price predictions, analyze airline and route patterns, and predict passenger satisfaction.

I used **Python, Pandas, NumPy, Scikit-Learn, Plotly and Streamlit**. For flight-price prediction, I use a regression approach, while passenger satisfaction is treated as a classification problem. I trained the models separately, saved them using Joblib, and integrated the trained models into a Streamlit application.

The user enters flight details in the application, the input is validated and transformed into the feature format expected by the trained model, and the model generates the prediction.”

Your application is structured around Home, Flight Price Prediction, Dashboard, Customer Satisfaction and About sections.

---

2. Why is flight price prediction regression?

Answer:

“Flight price prediction is a **regression problem because the target variable is continuous numerical data**. For example, the model could predict ₹4,500, ₹6,250 or ₹9,800. These are numerical values rather than predefined categories.

Therefore, instead of predicting a class, the model learns the relationship between flight features and the numerical ticket price.”

If examiner asks: *Why not classification?*

“We could convert prices into categories such as budget, average and premium, but that would lose the exact price information. Since my objective is to estimate the actual ticket price, regression is more appropriate.”

---

3. Why is satisfaction prediction classification?

Answer:

“Customer satisfaction is a classification problem because the output represents discrete categories, such as **satisfied or dissatisfied**. The model learns patterns in passenger and service-related features and assigns the passenger to one of the predefined classes.”

Remember:

Price → continuous → Regression

Satisfaction → category → Classification

---

4. Why Random Forest?

Answer:

"I selected Random Forest because the project involves **tabular data with both numerical and categorical information**, and the relationship between flight features and price may be nonlinear.

Random Forest combines multiple decision trees, which makes it more robust than relying on a single decision tree. It can capture nonlinear relationships and interactions between features and generally performs well on structured datasets."

Stronger technical answer:

"Another advantage is that Random Forest provides feature-importance information, which can help us understand which variables contribute more to the prediction."

If asked: *Why not Linear Regression?*

"Linear Regression assumes a linear relationship between the predictors and target. Flight prices can be affected by complex interactions between airline, route, timing, duration and stops, so a tree-based ensemble can capture nonlinear relationships more effectively."

---

5. How does Random Forest work?

Answer:

"Random Forest is an ensemble learning algorithm. Instead of building one decision tree, it builds multiple decision trees using randomized samples of the training data and randomized subsets of features.

Each tree makes a prediction. For regression, the predictions from the trees are generally averaged to produce the final prediction. For classification, the trees vote for a class, and the majority class becomes the final prediction.

Because the model combines many trees, it generally reduces the instability and overfitting associated with an individual decision tree."

Simple example:

Suppose 5 trees predict:

₹5,000, ₹5,500, ₹5,200, ₹5,300, ₹5,100

The Random Forest Regressor aggregates those predictions to produce the final estimate.

---

6. What feature engineering did you perform?

Answer:

"I performed feature engineering to convert the raw flight information into features that the trained model can use.

In particular, time-related information is transformed into numerical components such as **departure hour and departure minute**. I also convert stop information into a numerical representation such as **total number of stops**.

The important point is that the features created during prediction must follow the same schema and transformation logic that was used during model training."

Your application specifically checks the prediction DataFrame's columns, shape and data types, and the saved model's expected feature structure.

If asked: *Why is feature engineering important?*

"Because machine-learning algorithms operate on structured numerical representations. Good feature engineering converts raw business information into meaningful variables that the model can learn from."

---

7. What caused the missing-column error?

This is **one of the most important questions for you**, because you actually encountered this problem.

Answer:

"The error occurred because there was a mismatch between the **features expected by the trained model and the columns being supplied by my prediction DataFrame**.

The model expected features such as Dep_Hour, Dep_Minute and Total_Stops_Number, but those columns were initially missing from the prediction input.

Therefore, Scikit-Learn could not apply the trained preprocessing/model pipeline correctly and raised a missing-column error."

Very important:

Don't say:

"Python had an error."

Say:

"There was a mismatch between the training feature schema and the inference feature schema."

That sounds much more technically accurate.

---

8. How did you debug the model-input mismatch?

Answer:

"I debugged it by inspecting the prediction DataFrame before sending it to the model. I checked the **column names, shape and data types**.

Then I inspected the saved model to identify the features it expected, including feature_names_in_ and the pipeline steps.

I compared the model's expected schema with the application's prediction schema and identified the missing features. I then aligned the feature-engineering process and prediction DataFrame with the trained model's expected input."

Your application now contains a dedicated **Flight Model Input Validation** section that displays model input columns, shape and data types.

It also inspects the saved model's feature_names_in_ and pipeline structure.

If examiner asks: *What did you learn from this error?*

Excellent answer:

"I learned that in machine-learning deployment, **training-time preprocessing and inference-time preprocessing must be consistent**. A model can be correctly trained but still fail during deployment if the application sends a different feature schema."

---

9. What is RMSE and why did you use it?

Answer:

"RMSE stands for **Root Mean Squared Error**. It measures the difference between actual values and predicted values in a regression problem.

I used RMSE because flight price prediction is a regression task. RMSE is useful because it penalizes larger errors more strongly than smaller errors.

Also, RMSE is expressed in the same unit as the target variable, so in this case it can be interpreted in terms of the flight-price value."

The conceptual formula is:

If examiner asks: *Is lower or higher RMSE better?*

"Lower RMSE is better because it means the predictions are, on average, closer to the actual values."

If asked: *RMSE vs MAE?*

"Both measure prediction error, but RMSE squares the errors before averaging, so it gives greater importance to large errors. MAE uses absolute errors and is less sensitive to large outliers."

---

10. How does your Streamlit application work from user input → prediction?

This is another **very important answer** to practice.

Answer:

"First, the user enters the flight information through the Streamlit interface.

Then the application validates the input, including checking whether the selected route exists in the dataset.

Next, the application performs the required feature engineering. For example, time information is converted into the numerical features expected by the model, and stop information is transformed into the appropriate numerical representation.

After that, I create a Pandas DataFrame containing the prediction features in the required schema.

The DataFrame is passed to the saved machine-learning pipeline using the model's .predict() method.

The model returns the predicted ticket price. I then validate the output, prevent an invalid negative price, categorize the result as Budget, Average or Premium, and display the prediction in the Streamlit interface."

Your code explicitly calls the trained model's .predict() method and converts the result into the displayed prediction.

The application then categorizes the predicted price into Budget, Average or Premium fare levels.

⭐ **One-minute answer connecting all 10 questions**

If the examiner says **"Explain your technical implementation"**, you can give this answer:

"SkyPredict AI is an end-to-end machine-learning application for flight analytics. I use historical flight data to predict ticket prices and passenger data to predict customer satisfaction.

Flight price is a regression problem because the target is a continuous numerical value, whereas customer satisfaction is classification because the output consists of discrete classes.

I use Random Forest because it is an ensemble algorithm suitable for structured tabular data and can capture nonlinear relationships. I perform feature engineering to transform raw flight information into model-compatible features such as departure-hour, departure-minute and numerical stop information.

During deployment, I encountered a feature-schema mismatch where the trained model expected columns such as Dep_Hour, Dep_Minute and Total_Stops_Number, but they were initially missing from the prediction DataFrame. I debugged this by inspecting the DataFrame's columns, shape and data types and comparing them with the saved model's expected features.

Finally, the Streamlit application takes user input, validates and transforms it, creates the correct DataFrame, sends it to the saved model through .predict(), and displays the predicted ticket price. I evaluate the regression model using metrics such as RMSE, where a lower value indicates smaller prediction error."

This is the version I would practice for your viva. It demonstrates that you understand the **entire ML lifecycle**, rather than just knowing definitions.